

Contents

Guida al Codice - Sleep Apnea Detection	1
Cosa fa questo progetto?	2
Risultati principali	2
Struttura del Codice	2
Script Principali - Cosa Fanno	2
[1] prepare_dataset.py - Preparazione Dataset	2
[2] run_complete_analysis.py - Analisi Completa Standard	3
[3] prepare_dataset_manual.py - Prepara Dataset Manuale	4
[4] run_analysis_manual.py - Analisi con Parametri OTTIMIZZATI	4
[5] hyperparameter_tuning.py - Ricerca Automatica Hyperparametri	4
[6] visualize_tuning_results.py - Visualizza Risultati Tuning	5
Parametri Principali - Dove Trovarli e Come Modificarli	5
1. Lunghezza Finestre di Analisi (30 secondi)	5
2. Finestra Temporale Rolling (1.5 min → 3.5 min OTTIMALE)	6
3. Numero di Feature Selezionate (K)	6
4. Soglia di Classificazione	6
5. Hyperparametri Random Forest	6
6. Hyperparametri XGBoost	6
Come Esplorare Nuovi Hyperparametri - Workflow Pratico	7
Opzione 1: Grid Search Automatico (CONSIGLIATO)	7
Opzione 2: Test Manuale Singola Configurazione	7
Capire i Risultati	8
Metriche Principali:	8
Interpretazione:	8
Setup e Installazione	8
Esperimenti Suggesti	8
1. Testare Finestre Temporali Diverse	8
2. Testare Più Feature	9
3. Soglie Molto Basse per Screening	9
4. Ensemble RF + XGBoost	9
5. Validare su Altri Dataset	9
File di Output	9
Dataset Processati	9
Risultati Hyperparameter Tuning	9
Grafici e Visualizzazioni	9
Domande Frequenti	10
Q: Come cambio la lunghezza delle finestre da 30 secondi?	10
Q: Dove modifico il numero di alberi del Random Forest?	10
Q: Come posso testare 100 configurazioni invece di 480?	10
Q: I risultati non migliorano, cosa faccio?	10
Glossario Termini Tecnici	10

Guida al Codice - Sleep Apnea Detection

Guida per spiegare il progetto a un pubblico non esperto

Cosa fa questo progetto?

Questo progetto rileva automaticamente episodi di **apnea notturna** (quando una persona smette di respirare durante il sonno) usando: - Uno **smart mattress cover con 40 sensori di pressione** (griglia 4×10) che registra a 8 Hz come il paziente si muove durante la respirazione - Un **accelerometro** (6 canali) che misura i movimenti - Algoritmi di **Machine Learning** (Random Forest e XGBoost) per classificare automaticamente se un intervallo di 30 secondi contiene apnea o respirazione normale

Risultati principali

Il progetto ha raggiunto **ROC-AUC di 0.901** (su scala 0-1, dove 1 è perfetto) nel rilevare apnee, con **85% di recall**, usando un dataset etichettato manualmente con parametri ottimizzati.

Struttura del Codice

```
sleep-apnea-detection/
  README.md                # Documentazione principale del progetto
  requirements.txt          # Librerie Python necessarie
  data/                    # Dati grezzi (non inclusi nella repo)
  src/                     # Codice sorgente principale
    prepare_dataset.py      # [1] Prepara i dati dalle registrazioni grezze
    run_complete_analysis.py # [2] Esegue analisi completa con parametri standard
    cross_validation_analysis.py
    nested_cross_validation.py
    compare_all_approaches.py
  manual_labels/           # Script per dataset con etichette manuali
    prepare_dataset_manual.py # [3] Prepara dataset manuale
    run_analysis_manual.py    # [4] Analisi con parametri OTTIMIZZATI
    hyperparameter_tuning.py  # [5] Ricerca hyperparametri
    visualize_tuning_results.py # [6] Visualizza risultati tuning
  results/                 # Grafici e risultati salvati
  docs/                   # Documentazione aggiuntiva
```

Script Principali - Cosa Fanno

[1] prepare_dataset.py - Preparazione Dataset

Percorso: src/prepare_dataset.py

Cosa fa: Trasforma i dati grezzi delle registrazioni dello smart mattress cover in un dataset pronto per il machine learning.

Pipeline in 4 passi: 1. **Crea finestre di 30 secondi** dai segnali grezzi 2. **Estrae caratteristiche statistiche** da ogni finestra (media, deviazione standard, variabilità) 3. **Aggrega i 40 canali in 4 zone anatomiche** (torace sx/dx, addome sx/dx) 4. **Combina dati smart mattress cover + accelerometro**

Output: File CSV con ~50 caratteristiche per finestra di 30 secondi

Parametri Configurabili (linee 74-81):

```
# Sensor parameters
FS = 8.0          # Frequenza campionamento (Hz) - NON MODIFICARE
WINDOW_SEC = 30   # Durata finestra (secondi) - MODIFICABILE
WIN_SIZE = int(FS * WINDOW_SEC) # Campioni per finestra (calcolato)
N_CHANNELS = 40    # Numero canali smart mattress cover - NON MODIFICARE

# Labeling parameters
MIN_FRAC_APNEA = 0.5 # MODIFICABILE: frazione minima di apnea per etichettare finestra come apnea
MIN_FRAC_NONAPNEA = 0.5 # MODIFICABILE: frazione minima respiro normale
```

Come cambiare la lunghezza delle finestre: - Modifica WINDOW_SEC = 30 → prova 20, 40, 60 secondi
- Finestre più corte = più dettaglio ma più rumore - Finestre più lunghe = più stabile ma meno reattivo

[2] run_complete_analysis.py - Analisi Completa Standard

Percorso: src/run_complete_analysis.py

Cosa fa: Esegue l'intera pipeline di analisi con i parametri standard (dataset originale con etichette automatiche).

Fasi: 1. Carica dataset preparato 2. Separa train/test (divide per soggetti) 3. Addestra **modello baseline** (solo caratteristiche statiche) 4. Aggiunge **caratteristiche temporali** (delta, rolling mean/std, trend) 5. Addestra **Random Forest + XGBoost** con caratteristiche temporali 6. Esegue **cross-validation a 5 fold** 7. Genera grafici (ROC curves, confusion matrix, feature importance)

Parametri Configurabili (linee 36-40):

```
# Configuration
RANDOM_STATE = 42          # Seed random (per riproducibilità)
TEST_SUBJECTS = ['GZ01', 'FDR01', 'AM01', 'FC01'] # Soggetti test
N_ESTIMATORS = 400        # Numero alberi Random Forest
MIN_SAMPLES_LEAF = 3      # Minimo campioni per foglia RF
K_FEATURES = 60           # Numero feature selezionate
```

Come esplorare nuovi hyperparametri: - N_ESTIMATORS: prova 200, 400, 600, 800 (più alberi = più tempo ma potenzialmente migliore) - MIN_SAMPLES_LEAF: prova 1, 3, 5, 10 (valori più alti = meno overfitting) - K_FEATURES: prova 40, 50, 60, 70, 80 (quante feature mantenere dopo selezione)

Caratteristiche Temporali (linea ~150-180): **ROLLING_WINDOW = 3** (default):

```
def add_temporal_features(df, window_size=3):
    """
    Aggiunge caratteristiche temporali che catturano la dinamica respiratoria

    window_size = 3 finestre → 3 × 30s = 1.5 minuti
    """
    # Delta: variazione rispetto finestra precedente
    # Rolling mean: media mobile su window_size finestre
    # Rolling std: deviazione standard mobile
    # Trend: differenza tra valore corrente e rolling mean
```

Per modificare la finestra temporale: - Cerca `window_size=3` → cambia in 5 o 7 - Finestra più lunga (7) = cattura cicli respiratori più lunghi - **Risultato ottimale trovato: `window_size=7` (3.5 minuti)**

[3] `prepare_dataset_manual.py` - Prepara Dataset Manuale

Percorso: `src/manual_labels/prepare_dataset_manual.py`

Cosa fa: Versione specializzata di `prepare_dataset.py` per il dataset con etichette manuali (più accurate, +72% eventi apnea rispetto a etichette automatiche).

Identico a [1] ma legge file con suffisso `_indiciApneaManuale.csv`

[4] `run_analysis_manual.py` - Analisi con Parametri OTTIMIZZATI

Percorso: `src/manual_labels/run_analysis_manual.py`

Cosa fa: Stesso di [2] ma usa i **parametri ottimizzati** trovati tramite hyperparameter tuning.

Parametri OTTIMIZZATI (linee 48-52):

```
# PARAMETRI OTTIMIZZATI (da hyperparameter_tuning.py)
K_FEATURES = 70          # era 60 → +2% ROC-AUC
ROLLING_WINDOW = 7       # era 3 → +11% ROC-AUC (finestra 3.5 min)
CLASSIFICATION_THR = 0.25 # era 0.30 → +15% Recall
```

Risultati: ROC-AUC 0.901 ± 0.034 (vs 0.827 con parametri standard)

Perché questi parametri funzionano meglio? - `K_FEATURES=70`: mantiene più informazioni discriminanti - `ROLLING_WINDOW=7`: cattura cicli respiratori più lunghi (apnee durano 10-30 secondi) - `CLASSIFICATION_THR=0.25`: soglia più bassa → rileva più apnee (recall più alto)

[5] `hyperparameter_tuning.py` - Ricerca Automatica Hyperparametri

Percorso: `src/manual_labels/hyperparameter_tuning.py`

Cosa fa: Testa **480 configurazioni diverse** di hyperparametri per trovare la combinazione migliore.

Hyperparametri esplorati (linee ~165-175):

```
# Grid search parameters
K_VALUES = [40, 50, 60, 70, 80]
WINDOW_SIZES = [3, 5, 7]
N_ESTIMATORS_VALUES = [200, 400, 600]
THRESHOLDS = [0.25, 0.30, 0.35, 0.40]
XGB_MAX Depths = [4, 6, 8]

# Feature selection
# Rolling window (finestra temporale)
# Numero alberi RF
# Soglia classificazione
# Profondità alberi XGBoost
```

Come esplorare nuovi hyperparametri:

1. Aggiungi nuovi valori alle liste:

```
K_VALUES = [40, 50, 60, 70, 80, 90, 100] # + aggiungi 90, 100
WINDOW_SIZES = [3, 5, 7, 9, 11] # + aggiungi 9, 11 (4.5 e 5.5 min)
THRESHOLDS = [0.20, 0.25, 0.30, 0.35] # + aggiungi soglie diverse
```

2. Esegui lo script:

```
cd src/manual_labels
python hyperparameter_tuning.py
```

3. I risultati vengono salvati in: tuning_results/tuning_results_all.csv

4. Visualizza i risultati:

```
python visualize_tuning_results.py
```

Funzione Chiave - Cross-Validation (linee 85-150):

```
def run_cv(X_full, y_full, subjects_full, temporal_full,
          k_features, n_estimators, model_type='rf',
          xgb_max_depth=6, threshold=0.30, n_splits=5):
    """
    Esegue cross-validation con una specifica configurazione

    Parametri:
    - k_features: numero feature da selezionare
    - n_estimators: numero alberi
    - model_type: 'rf' o 'xgb'
    - xgb_max_depth: profondità XGBoost
    - threshold: soglia classificazione binaria
    - n_splits: fold cross-validation

    Output: ROC-AUC medio, std, F1, precision, recall
    """
```

[6] visualize_tuning_results.py - Visualizza Risultati Tuning

Percorso: src/manual_labels/visualize_tuning_results.py

Cosa fa: Genera grafici dalle 480 configurazioni testate per capire l'impatto di ogni hyperparametro.

Grafici generati: 1. **Window Size Effect:** come la finestra temporale influenza performance 2. **K Features Effect:** impatto del numero di feature 3. **Heatmap K vs Window:** interazione tra K e finestra temporale 4. **Threshold Analysis:** come la soglia influenza precision/recall trade-off 5. **N Estimators:** effetto numero alberi 6. **Optimization Summary:** confronto configurazioni migliori

Parametri Principali - Dove Trovarli e Come Modificarli

1. Lunghezza Finestre di Analisi (30 secondi)

src/prepare_dataset.py, linea 76

```
WINDOW_SEC = 30 # Cambia qui (es: 20, 40, 60)
```

Impatto: - Finestre più corte = più granularità, più rumore - Finestre più lunghe = più stabile, meno dettaglio

2. Finestra Temporale Rolling (1.5 min → 3.5 min OTTIMALE)

src/run_complete_analysis.py, linea ~150 (funzione add_temporal_features) src/manual_labels/run_analysis_manual.py, linea 50

ROLLING_WINDOW = 7 # Cambia qui (3=1.5min, 5=2.5min, 7=3.5min, 9=4.5min)

Impatto: - Finestre brevi (3) = cattura cambiamenti rapidi - Finestre lunghe (7-9) = cattura pattern respiratori completi - **Ottimale trovato: 7 (3.5 minuti)**

3. Numero di Feature Selezionate (K)

src/run_complete_analysis.py, linea 40 src/manual_labels/run_analysis_manual.py, linea 48

K_FEATURES = 70 # Cambia qui (40, 50, 60, 70, 80, 100)

Impatto: - K basso = modello più semplice, rischio underfitting - K alto = più informazioni, rischio overfitting - **Ottimale trovato: 70**

4. Soglia di Classificazione

src/manual_labels/run_analysis_manual.py, linea 51 src/run_complete_analysis.py, linea ~270

CLASSIFICATION_THR = 0.25 # Cambia qui (0.20-0.50)

Impatto: - Soglia bassa (0.20-0.25) = rileva più apnee (recall↑), più falsi positivi - Soglia alta (0.35-0.50) = rileva meno apnee (precision↑), più falsi negativi - **Trade-off precision vs recall**

5. Hyperparametri Random Forest

src/run_complete_analysis.py, linee 38-40

N_ESTIMATORS = 400 # Numero alberi (200, 400, 600, 800)
MIN_SAMPLES_LEAF = 3 # Campioni minimi per foglia (1, 3, 5, 10)

Impatto: - N_ESTIMATORS = modello più robusto, tempo training - MIN_SAMPLES_LEAF = meno overfitting, modello più semplice

6. Hyperparametri XGBoost

src/run_complete_analysis.py, linea ~310

```
xgb_model = xgb.XGBClassifier(  
    n_estimators=400,      # Numero alberi  
    max_depth=6,          # Profondità alberi (4, 6, 8, 10)  
    learning_rate=0.1,     # Tasso apprendimento (0.01, 0.1, 0.3)
```

```
    random_state=42
)
```

Impatto: - `max_depth` = alberi più complessi, rischio overfitting - `learning_rate` = apprendimento più lento ma stabile

Come Esplorare Nuovi Hyperparametri - Workflow Pratico

Opzione 1: Grid Search Automatico (CONSIGLIATO)

1. **Apri:** `src/manual_labels/hyperparameter_tuning.py`

2. **Modifica le griglie di ricerca** (linee ~165-175):

```
# Esempio: esplora finestre temporali più lunghe
K_VALUES = [60, 70, 80, 90]           # Aggiungi 90
WINDOW_SIZES = [5, 7, 9, 11]         # Aggiungi 9, 11 (4.5 e 5.5 min)
N_ESTIMATORS_VALUES = [400, 600]     # Riduci per salvare tempo
THRESHOLDS = [0.20, 0.25, 0.30]      # Prova soglie più basse
XGB_MAX Depths = [6, 8]              # Profondità maggiore
```

3. **Esegui:**

```
cd src/manual_labels
python hyperparameter_tuning.py
```

4. **Visualizza risultati:**

```
python visualize_tuning_results.py
```

5. **Analizza:**

- Apri file: `tuning_results/tuning_results_all.csv`
 - Ordina per `roc_auc_mean` decrescente
 - Trova la configurazione migliore
 - Guarda i grafici in `tuning_figures/`
-

Opzione 2: Test Manuale Singola Configurazione

1. **Apri:** `src/manual_labels/run_analysis_manual.py`

2. **Modifica parametri** (linee 48-51):

```
K_FEATURES = 80           # Prova 80 invece di 70
ROLLING_WINDOW = 9        # Prova 9 (4.5 minuti)
CLASSIFICATION_THR = 0.22 # Prova soglia più bassa
```

3. **Esegui:**

```
cd src/manual_labels
python run_analysis_manual.py
```

4. **Confronta risultati:**

- ROC-AUC
- F1-Score

- Precision vs Recall trade-off

Capire i Risultati

Metriche Principali:

Metrica	Cosa Misura	Valore Ottimo
ROC-AUC	Capacità generale di discriminare apnea/non-apnea	0.90-1.00 (eccellente)
Recall	% di apnee realmente rilevate	80-90% (obiettivo clinico)
Precision	% di predizioni “apnea” corrette	40-60% (accettabile per screening)
F1-Score	Bilanciamento precision/recall	0.50-0.70 (buono)

Interpretazione:

- **ROC-AUC 0.90+**: Modello eccellente
- **Recall 85%**: Rileva 85 apnee su 100 (15% falsi negativi)
- **Precision 48%**: Metà degli allarmi sono veri (trade-off accettabile per screening)

Setup e Installazione

```
# 1. Installa dipendenze
pip install -r requirements.txt

# 2. Prepara dataset (se hai dati grezzi)
cd src
python prepare_dataset.py

# 3. Esegui analisi
python run_complete_analysis.py

# 4. O con parametri ottimizzati (dataset manuale)
cd manual_labels
python run_analysis_manual.py

# 5. Per hyperparameter tuning
python hyperparameter_tuning.py
python visualize_tuning_results.py
```

Esperimenti Suggestiti

1. Testare Finestre Temporali Diverse

```
# In src/manual_labels/hyperparameter_tuning.py
WINDOW_SIZES = [3, 5, 7, 9, 11, 13] # Da 1.5 a 6.5 minuti
```

Domanda: C'è un plateau oltre 7? Finestre ancora più lunghe migliorano?

2. Testare Più Feature

```
K_VALUES = [60, 70, 80, 90, 100, 120]
```

Domanda: L'ottimale è davvero 70 o c'è margine di miglioramento?

3. Soglie Molto Basse per Screening

```
THRESHOLDS = [0.15, 0.18, 0.20, 0.22, 0.25]
```

Obiettivo: Massimizzare recall (rileva quasi tutte le apnee) per uno screening iniziale

4. Ensemble RF + XGBoost

Modifica `run_analysis_manual.py` per combinare le predizioni:

```
# Dopo training di entrambi i modelli  
y_pred_ensemble = (y_pred_proba_rf + y_pred_proba_xgb) / 2
```

5. Validare su Altri Dataset

Cambia soggetti di test:

```
TEST_SUBJECTS = ['CM01', 'GZ01'] # Prova diverse combinazioni
```

File di Output

Dataset Processati

- `dataset_windows_30s_features_zones_CORRECTED.csv`
Dove: `preprocessing_output/`
Contenuto: Dataset processato pronto per machine learning

Risultati Hyperparameter Tuning

- `tuning_results_all.csv`
Dove: `tuning_results/`
Contenuto: Tutte le 480 configurazioni testate

Grafici e Visualizzazioni

- `roc_curves_CORRECTED.png`
Dove: `results/`
Contenuto: Curve ROC comparative tra modelli
- `confusion_matrices_CORRECTED.png`
Dove: `results/`
Contenuto: Matrici di confusione

- `cv_boxplot.png`
Dove: `results/manual_labels/`
Contenuto: Distribuzione risultati cross-validation
-

Domande Frequenti

Q: Come cambio la lunghezza delle finestre da 30 secondi?

A: Modifica `WINDOW_SEC` in `prepare_dataset.py` (linea 76) e ri-esegui la preparazione dataset.

Q: Dove modifico il numero di alberi del Random Forest?

A: `N_ESTIMATORS` in `run_complete_analysis.py` (linea 38) o `run_analysis_manual.py`.

Q: Come posso testare 100 configurazioni invece di 480?

A: Riduci le griglie in `hyperparameter_tuning.py`:

```
K_VALUES = [60, 70]           # Solo 2 valori
WINDOW_SIZES = [5, 7]         # Solo 2 valori
N_ESTIMATORS_VALUES = [400]   # Solo 1 valore
# ecc...
```

Q: I risultati non migliorano, cosa faccio?

A: 1. Verifica che il dataset sia bilanciato (oversampling attivo?) 2. Prova a visualizzare feature importance 3. Controlla se ci sono NaN o outlier nei dati 4. Considera che potresti aver già raggiunto il limite dei dati disponibili

Glossario Termini Tecnici

- **ROC-AUC:** Area sotto la curva ROC (0-1, più alto = migliore)
 - **Recall:** Sensibilità, quante apnee reali vengono rilevate
 - **Precision:** Quante predizioni “apnea” sono corrette
 - **Cross-validation:** Valida il modello su più sottogruppi per evitare overfitting
 - **Temporal features:** Caratteristiche che catturano come i segnali cambiano nel tempo
 - **Oversampling:** Replica esempi di apnea per bilanciare il dataset
 - **Feature selection:** Seleziona le K feature più discriminanti
 - **GroupKFold:** Cross-validation che raggruppa per soggetto (evita data leakage)
-

Ultima modifica: Febbraio 2026

Autore: Documentazione generata per spiegazione progetto PhD